

[illegible]

I used a total of 60 states. Every state where MAR gets loaded, I enter my translation sequence of states that translates virtual addresses to physical addresses. This includes states 18/19, 2, 6, 7, 3, 15, etc. I check for unaligned exceptions when an address is generated before the page walk, which will go to state 10 (my default exception state) if an odd address is detected. If not, throughout the rest of the set of states, I calculate the PTE address, load the PTE, check for protection exceptions and page faults (go to state 10 if detected), set reference and sometimes modified bit (when write), and calculate the physical address. After the page walk, it returns to the state after when the translation occurred (Jnext) depending on what the original state was. (explained more in depth later) For example, if state=18/19, Jnext=33. I edited the states within the interrupt/exception service routine to prevent the MDR getting lost when translation happens with a Saved_MDR register.

[illegible]

Saved_MDR register holds the PSR/PC so that when a page walk occurs in the interrupt/exception routine states, these values that were in the MDR don't get destroyed. To set the reference and modified bit of the MDR, my PTE bit logic block always sets the R bits, but checks if the instruction is an STB/STW (signal MBIT = 1) to set the M bit. My exception vector generating logic sets EXCV to x03, x04, x02, or x05 depending on inputs of MDR (PTE), PSR[15], MAR, IR[15:12], and STATE_NUMBER. New control signals I added include:

- Extending MARMUX to 2 bits to also select from calculating PTE address or physical address in the translation
- LD.VA and LD.JNEXT to load the registers that are necessary in the translation
- GatePTE to update the MDR after setting the reference and/or modified bit
- MBIT, high if instruction is a STW or STB so we know to set the modified bit
- TRANS, high at the end of translation to return back to the correct state

I added an AND gate that evaluates if STORE is a 1 (high during an STW/STB instruction). This gets ORed with J[4] to get the next state number that will set the modified bit of the PTE. My exceptions, interrupts and returning from translation gets handled by a mux I added that chooses from either the originally calculated J bits, state 10 (exception), state 36 (interrupt), or Jnext(return after translation). Since exceptions have highest priority, if the exception flag is high, it will always go to state 10. Otherwise, if the interrupt flag is high, it'll go to state 36. If both these flags are low, it'll check my TRANS (translation) flag that routes the next state to whatever is in the Jnext register after the page walk. Lastly, if all three flags are low, it goes to the J bits.

